
Recursive Language Models

The cutting edge of context processing, and the real-world debate that proves it matters.

Recursive Language Models (RLM)

Alex Zhang, Tim Kraska & Omar Khattab — MIT CSAIL

Paper: arxiv.org/abs/2512.24601

Traditional LLM

- The prompt **IS** the input to the network.
- Every token goes through attention.
- Result: **Context rot happens.**

RLM Paradigm

- Prompt is stored as a **VARIABLE**.
- Model interacts with it via **CODE**.
- Result: **No context rot. Infinite scale.**

How RLM Works — The Architecture

1. ROOT LLM

- Receives **only the query**.
- Does not see the massive input directly.
- Acts as the "Controller" or "Researcher".

2. REPL ENVIRONMENT

- Stores input as a **Python variable**.
- Executes code written by the Root LLM.
- Returns results (slices, searches) to the model.

3. AVAILABLE TOOLS

- `context[:1000]` (Peek)
- `re.findall(...)` (Search)
- `context[a:b]` (Slice)
- `rlm_agent(q, chunk)` (Recurse)

The Agentic Loop: Write Code → Execute → See Output → Iterate

Same pattern as agents, but the "tool" is its own context.

RLM Step-by-Step Example

Query: "What was ACME Corp's Q3 APAC revenue?"

Context: 500-page annual report (stored as Python variable)

Step 1

Initial Peek

Root LLM reads `context[:2000]` to find the Table of Contents and document structure.

Step 2

Targeted Search

Model runs `re.findall(r'APAC|Asia Pacific', context)` to locate relevant page offsets.

Step 3

Context Slicing

Model slices `context[45000:52000]` and calls a Sub-LLM to extract the specific Q3 figure.

Step 4

Final Synthesis

Root LLM receives the answer from the sub-agent and compiles the final response for the user.

The root model never processes the full 500 pages. Zero context rot.

The Power of Recursion

Root LLM (Depth 0)

└─ rlm_agent(query, chunk_1) [D1]

└─ rlm_agent(q, sub_1a) [D2]

└─ rlm_agent(q, sub_1b) [D2]

└─ rlm_agent(query, chunk_2) [D1]

└─ rlm_agent(q, sub_2a) [D2]

Human-like Research

1. **Scan:** Check Table of Contents / Index.
2. **Narrow:** Identify relevant chapters.
3. **Deep-dive:** Read specific sections carefully.
4. **Synthesize:** Combine findings into an answer.

Scalability: 10M tokens? 100M tokens? No problem.

Each level sees only a SMALL, focused piece. Results flow back up.

Why RLM Beats Everything

1. NO CONTEXT ROT

Each LLM call sees only a **small, focused piece**. The "lost in the middle" problem completely disappears.

2. INFINITE SCALE

10K tokens? 100M tokens? The approach remains the same —just **recurse deeper** into the data structure.

3. ACTIVE EXPLORATION

The model isn't passive. It **chooses its path**, filters noise via Python, and only reads what actually matters.

4. COST-EFFECTIVE

Irrelevant text is filtered by **string operations** (free), not by the transformer (expensive). Only "gold" tokens hit the GPU.

RLM Results — Mind-Blowing

Model Configuration		Performance Gain	Context Capacity
RLM(GPT-5-mini)	vs Vanilla GPT-5	+34.2 Points	100x Beyond Window
RLM(Qwen3-8B)	vs Base Qwen3-8B	+28.3% Average	Infinite Recursion
RLM(Llama-3-70B)	vs Vanilla Llama-3	+19.5 Points	Scales to 10M+ Tokens

"A mini model using RLM beats a full model with raw context."

Handles inputs up to 100x beyond model context window at comparable or lower cost.

Claude Code vs Cursor

The Great Code Search Debate

Claude Code

Agentic Search

Uses shell tools (grep, glob) to explore codebases actively.
No pre-computed index required.

Cursor

Indexed RAG

Uses embeddings and vector databases to map code semantically. Requires background indexing.

Both are billion-dollar products solving the same problem differently.

Claude Code — Agentic Search

The Toolkit

- ls
- glob
- grep
- cat

No Embeddings. No Index.

The Agentic Process

- 1. Discovery:** "What files exist?" → `ls`, `glob`
- 2. Navigation:** "Where is this function?" → `grep`
- 3. Inspection:** "What does it do?" → `cat`
- 4. Context:** "What calls it?" → `grep` references

Philosophy: Active exploration over passive consumption.

The model DECIDES the search strategy, iterating until it understands.

Why Agentic Search Works

1. PRECISION

exact matches

Code has exact symbols. `grep` provides with zero ambiguity. Embeddings often return "fuzzy positives"—similar but technically wrong results.

2. FRESHNESS

Code changes constantly. Agentic search reads the **actual current file**. No waiting for re-indexing, re-chunking, or re-embedding after every save.

3. SECURITY & SIMPLICITY

No index to build or maintain. No sensitive code snippets stored in **third-party vector databases**. Everything stays local to the agent's environment.

"Even for our own codebase [at Anthropic], it's very sensitive. We don't want to upload it to a third-party thing."

— Boris Cherny, Anthropic

Cursor — The Indexed RAG Approach

SMART CHUNKING

Uses **Tree-sitter** to parse code into ASTs. Chunks at syntactic boundaries (functions/classes) so logic is never split in half.

INCREMENTAL SYNC

Uses **Merkle trees** to detect exact file changes. Re-embeds only modified files instantly, keeping the index fresh.

TEAM REUSE

Clones share **92% similarity**. New team members can reuse existing teammate indexes, dropping setup time to near zero.

SEMANTIC SEARCH

Enables conceptual queries like *"Where do we handle rate limiting?"* without needing to know specific function names.

Result: Semantic search improved accuracy by 12.5%.

Heavy investment in making indexing fast, fresh, and secure.

Comparing the Struggles

CLAUDE CODE (AGENTIC)

- **Token Burn:** Grep results load directly into context, increasing costs.
- **Latency:** Sequential "20 questions" with the filesystem can be slow.
- **Conceptual Search:** Hard to find what you can't explicitly name.

CURSOR (INDEXED RAG)

- **Index Staleness:** 5-minute sync delay can lead to file drift.
- **Setup Time:** Large repositories can take hours to index initially.
- **Semantic Noise:** "Similar but wrong" results can distract the model.

Neither is perfect; they offer different trade-offs for different workflows.

The 2026 Consensus & Summary

THE BEST TEAMS USE BOTH

Claude Code: Deep, autonomous, agentic work. Best for complex multi-file refactors.

Cursor: Quick, visual, interactive work. Best for semantic discovery and rapid UI iteration.

THE CORE LESSON

How the model relates to context is the fundamental design decision of modern AI systems.

RLM, Claude Code, and Cursor are just different answers to the same problem.

The Paradigm Shift

"Understanding why each approach fits its domain is what separates someone who uses AI tools from someone who builds them."

Don't blindly stuff the context window.